深度优先遍历

对于树结构的搜索(或遍历)来讲, 分为深度优先(DFS)和广度优先(BFS).

```
function dfs(node) {
  console.log('探寻阶段: ', node.name);
  node.children.forEach(child => {
    dfs(child);
  });
  console.log('回溯阶段: ', node.name);
}
```

React 当中的使用场景

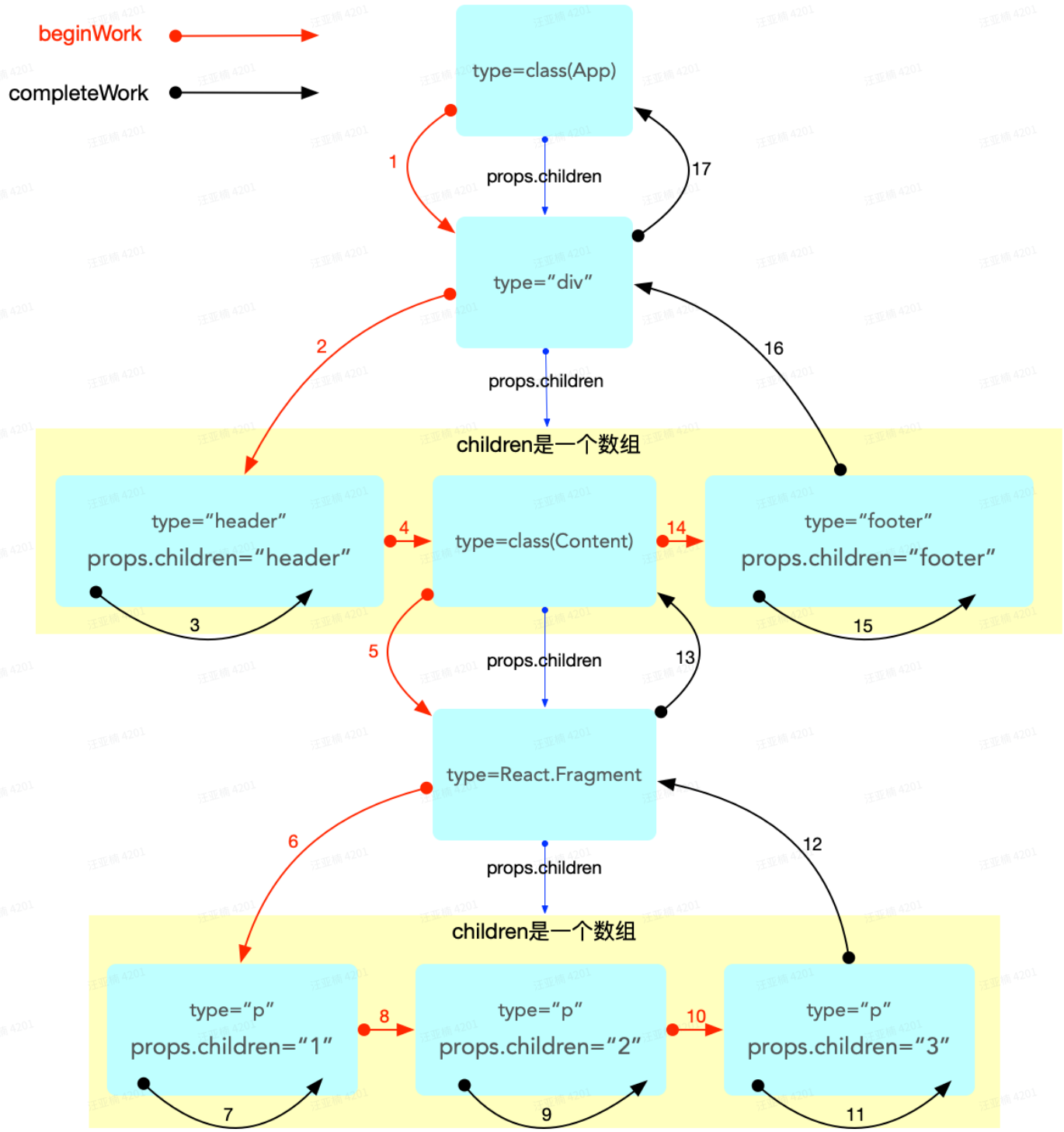
ReactElement "树"的构造

`ReactElement` 不能算是严格的树结构, 为了方便表述, 后文都称之为树.

```
class App extends React.Component {
  render() {
    return (
      <div className="app">
        <header>header</header>
        <Content />
        <footer>footer</footer>
      </div>
    );
  }
}
class Content extends React.Component {
  render() {
    return (
      <React.Fragment>
        <p>1</p>
        <p>2</p>
        <p>3</p>
      </React.Fragment>
    );
  }
}
```

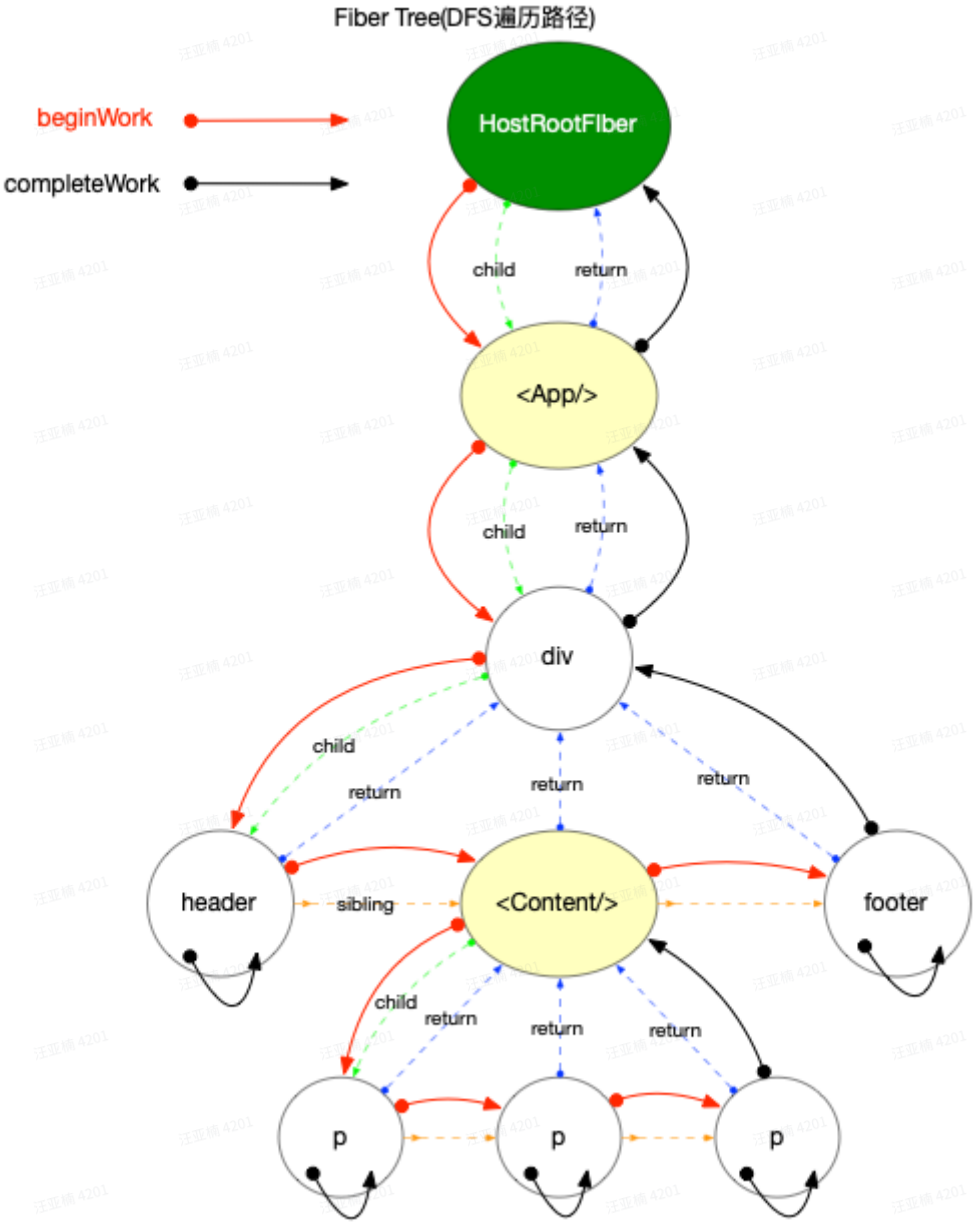
```
export default App;
```

ReactElement结构(DFS遍历路径)



注意:

- `ReactElement` 树是在大循环中的 `beginWork` 阶段"逐级"生成的.
- "逐级"中的每一级是指一个 `class` 或 `function` 类型的组件, 每调用一次 `render` 或执行一次 `function` 调用, 就会生成一批 `ReactElement` 节点.
- `ReactElement` 树的构造, 实际上就是各级组件 `render` 之后的总和.

fiber 树的构造

在 `ReactElement` 的构造过程中, 同时伴随着 `fiber` 树的构造, `fiber` 树同样也是在 `beginWork` 阶段生成的.